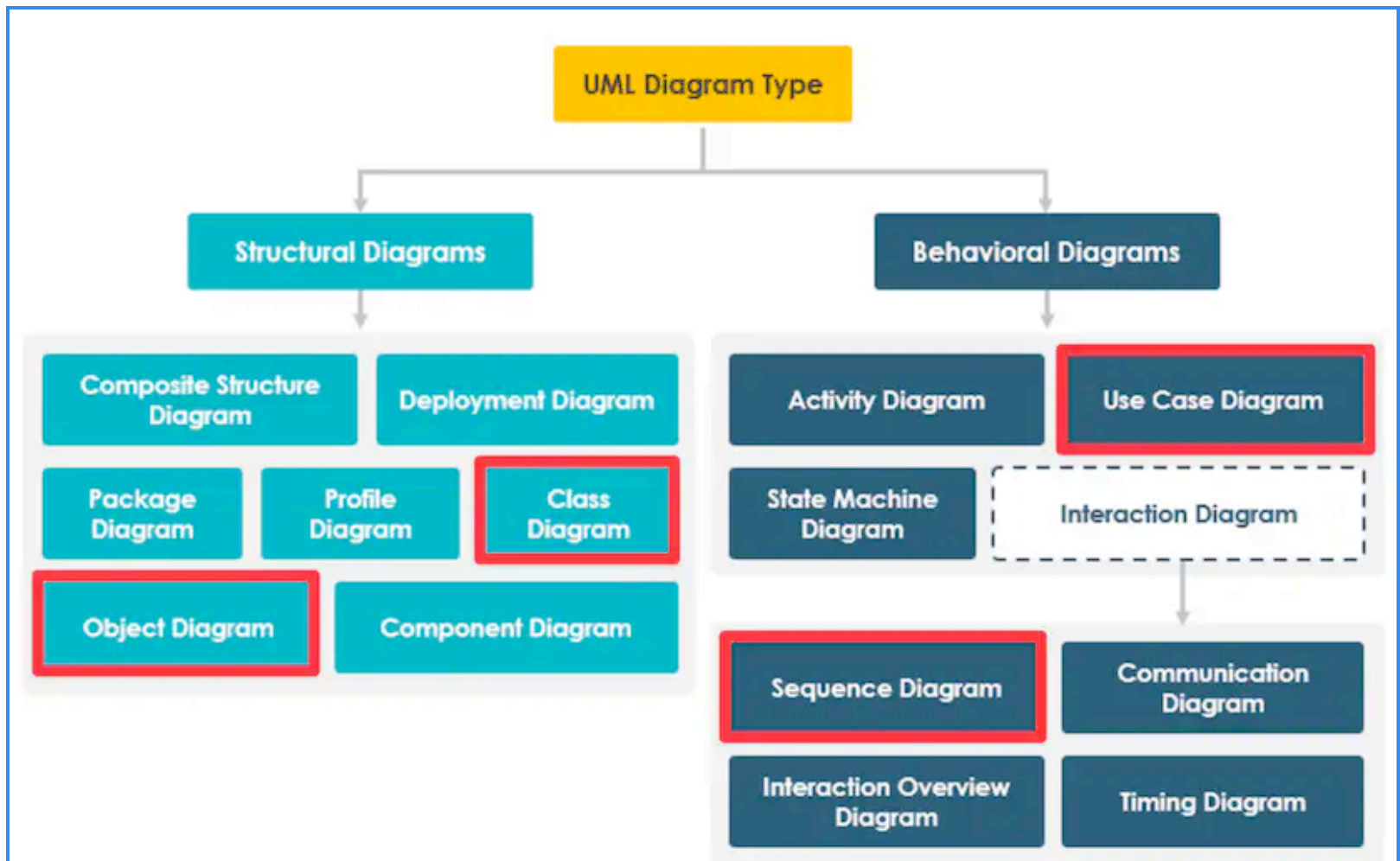


# UML Diagrams

Behavioral Diagrams

- So far, we have discussed "Structural Diagrams" in UML.
  - Class Diagram & Object Diagram and their relationships.
- Now, we discuss "Behavioral Diagrams".
  - Use Case Diagram & Sequence Diagram



# Use Case Diagram

A Use Case diagram captures functional requirements and system scope:

- **Actors:** Users or external systems outside the system boundary
- **System Boundary:** Scope of the system being built
- **Use Cases:** Specific functional goals the system performs for actors

For example, in a library system:

- We identify users and their key goals:
  - **Students** want to search and borrow books.
  - **Librarians** want to manage the catalog and user records.

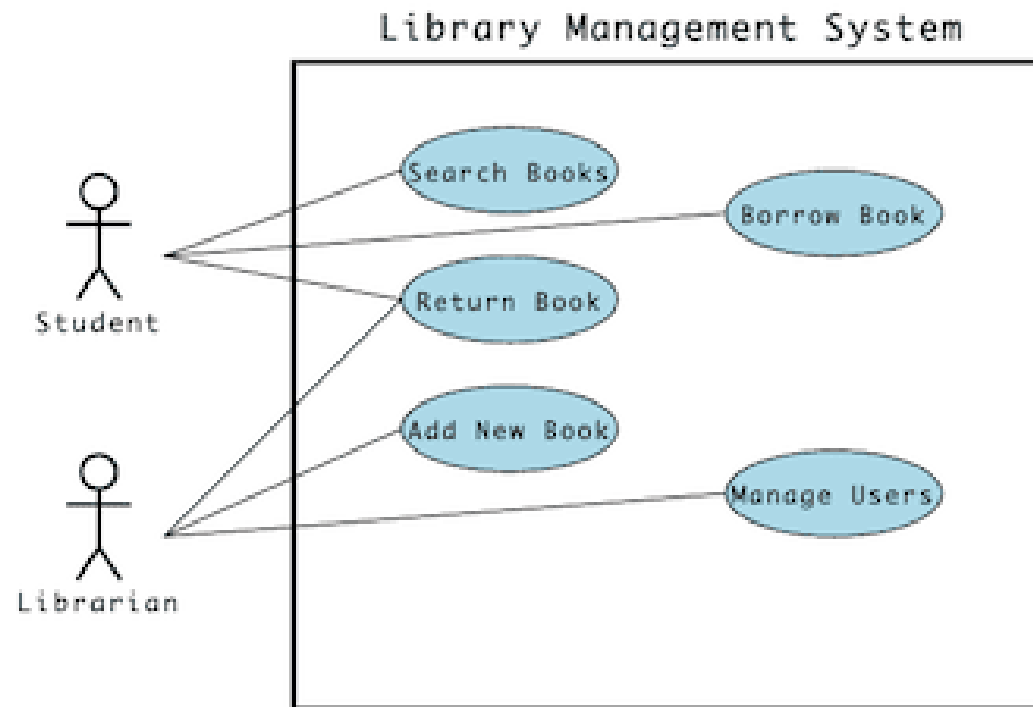
In Agile analysis, requirements often start with the **User Story** format:

- *"As a [role], I want to [goal], so that [benefit]"*
- *"As a student, I want to **search for books** so that I can find research materials."*

We apply the same format for the librarian:

- *"As a librarian, I want to **add new books** so that students can access an up-to-date catalog."*
- *"As a librarian, I want to **manage user accounts** so that I can keep student records current."*

Using these requirements, we construct a Use Case diagram:



## | Why Use Case Diagrams?

- Bridges communication between developers and non-technical stakeholders
- Clearly defines system scope and external boundaries
- Provides the basis for deriving acceptance test scenarios

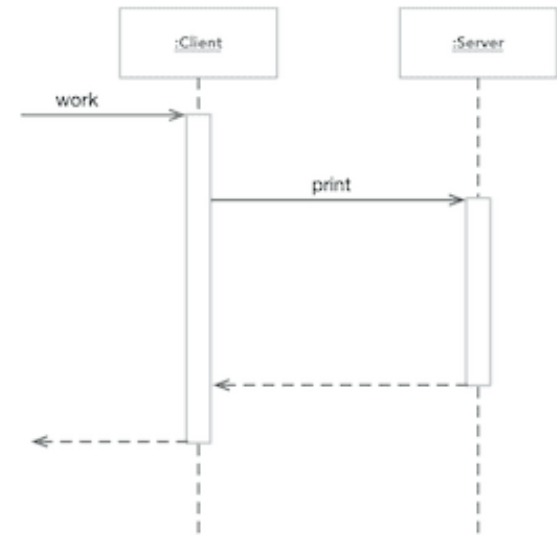
# Sequence Diagram

- UML sequence diagrams model object interactions and message flow over time.
- Example collaboration:
  - A `Client` object executes its `work()` method.
  - Inside `work()`, it calls `Server.print_document()`.

```
class Server:
    def print_document(self):
        print("Document printed.")

class Client:
    def work(self):
        server = Server()
        server.print_document()

c = Client()
c.work()
```



- Key Sequence Diagram notation:
  - **Lifeline:** Participant object represented as `<u>instance : Class</u>` with a dashed line extending downwards.
  - **Synchronous Message:** Solid line with filled arrowhead ( $\rightarrow$ ).
  - **Return / Reply Message:** Dashed line with open arrowhead ( $--\rightarrow$ ).
  - **Activation Box:** Vertical rectangle showing active execution.

## | Why Sequence Diagrams?

- Traces the chronological execution path across collaborating objects.
- Exposes design flaws early (e.g., circular dependencies, tight coupling, bottleneck objects).
- Aligns team members on object interactions and method contracts.